Mbau
Daris
RT31-ROM-FA

Jeudi, 11/Dec. 2025

Situation d'Apprentissage et d'Evaluation
SAE5.ROM.03 – Déployer et gérer les services ROM

Introduction

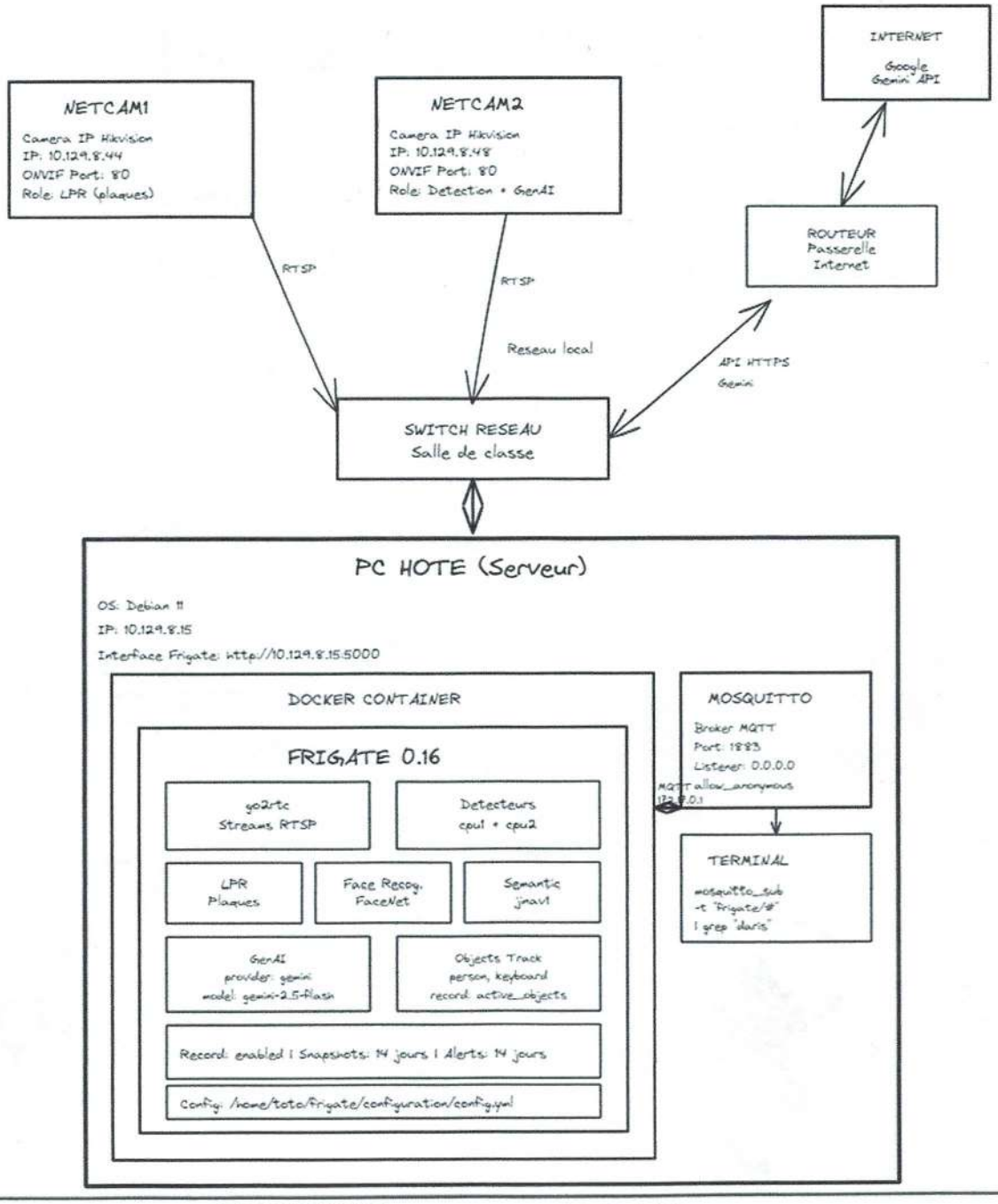
J'ai réalisé ce projet dans le cadre de la SAE5.ROM.03 dédiée au déploiement et à la gestion des services ROM. Mon objectif principal était de mettre en place un système de vidéosurveillance intelligent. On a choisit Frigate comme solution VMS (Vidéo Management System). Ce logiciel open-source m'a permis d'intégrer des fonctionnalités avancées d'analyse vidéo. L'architecture que j'ai déployée repose sur Docker. Cette technologie de conteneurisation facilite la gestion et l'isolation des services. Mon serveur Debian 12 héberge l'ensemble de l'infrastructure. Deux caméras IP Hikvision connectées en réseau alimentent le système. Elles communiquent via le protocole RTSP (Real Time Streaming Protocol). J'ai configuré ONVIF pour contrôler les caméras PTZ (Pan-Tilt-Zoom) à distance.

Le projet m'a amené à explorer plusieurs technologies d'intelligence artificielle. La détection d'objets utilise des modèles COCO préentraînés. Ces modèles reconnaissent 25 classes d'objets différentes. J'ai implémenté la lecture automatique de plaques d'immatriculation. le système LPR (License Plate Recognition) natif de Frigate analyse les caractères avec precision. La reconnaissance faciale s'appuie sur FaceNet. Ce réseau de neurones génère des embeddings pour identifier les visages. J'ai ajouté la recherche sémantique via Jina AI CLIP. Cette fonctionnalité permet des requêtes en langage naturel. L'IA générative Gemini complète le dispositif.

Chaque étape du projet suit une méthodologie rigoureuse. J'ai d'abord étudié la documentation de Frigate. Ensuite, j'ai installé et configuré l'environnement Docker.

La connexion des caméras a nécessité des ajustements réseau précis. les tests de détection ont validé le bon fonctionnement du système. L'intégration de l'IA a représenté le défi technique majeur.
Ce rapport détaille mes choix techniques, mes configurations et mes résultats. Il présente également les difficultés rencontrées et les solutions apportées.

Schéma d'infrastructure



CAMERAS:
- NetCam1: Lecture plaques (LPR)
  Source: fichier video
  /config/videos/
- NetCam2: Detection + GenAI
  Source: rtsp://admin:HikVision@IP

SERVICES FRIGATE:
- LPR: threshold 0.7/0.8
- Face Recognition: FaceNet small
  threshold 0.9
- Semantic Search: jinav1 small
- GenAI: Gemini 2.5 flash
  Prompts en francais

DETECTEURS:
- cpu1: 3 threads
- cpu2: 3 threads

MQTT:
- Topics: frigate/events, frigate/stats
- Filtrage: grep pour alertes

PORTS:
- Frigate Web: 5000
- MQTT: 1883
- RTSP: 554, 8554

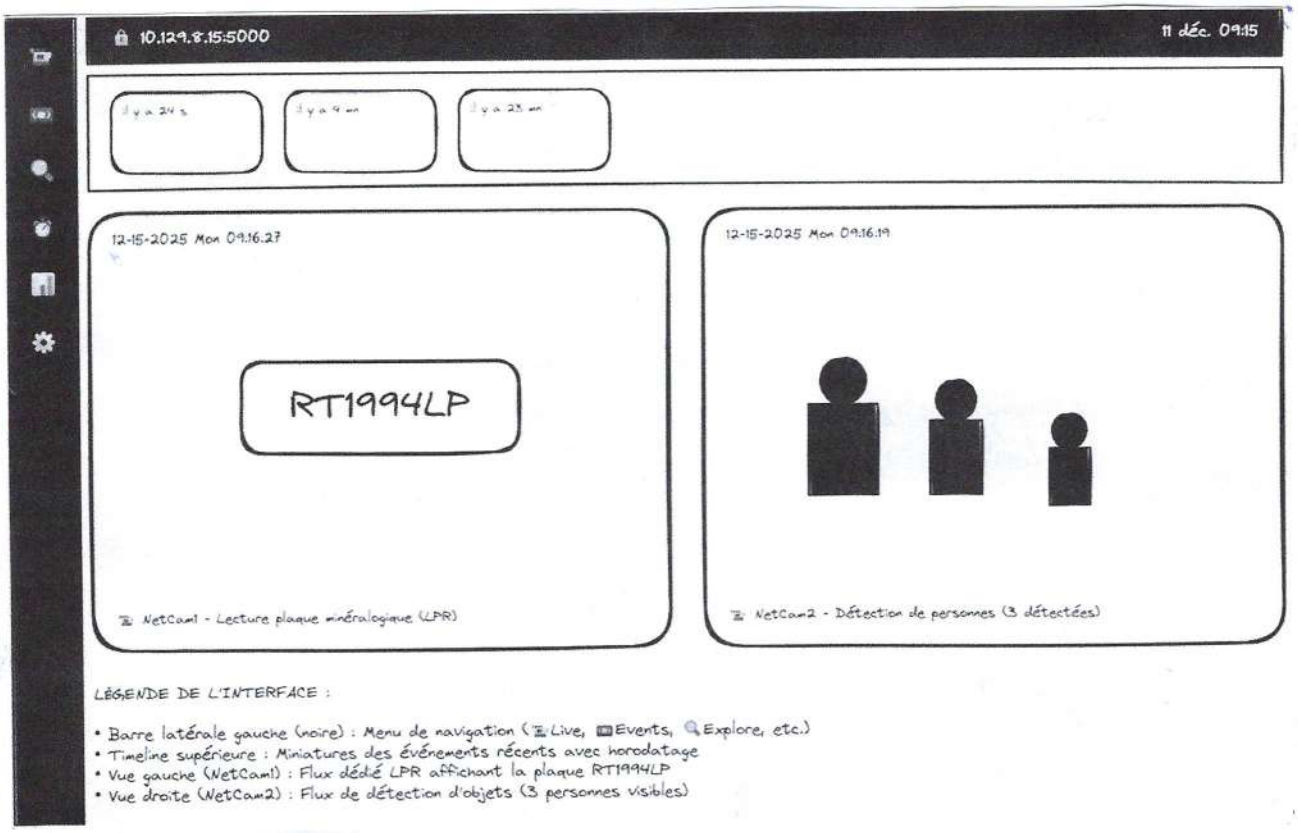
1. Etude de la documentation du VMS.

J'ai commencé par consulter la documentation officielle de Frigate. Ce VMS open-source m'a semblé adapté au projet. La documentation se trouve sur https://docs.frigate.video/. J'ai exploré plusieurs sections clés. La partie installation m'a guidé pour Docker. La section configuration détaille les fichiers YAML. Les exemples de caméras m'ont aidé à comprendre le RTSP. J'ai aussi étudié les capacités d'IA. Frigate intègre TensorFlow lite pour la detection. La reconnaissance faciale utilise FaceNet. La documentation explique aussi ONVIF pour contrôler les caméras PTZ.
La documentation recommande 8GB de RAM minimum. Mon serveur Debian 12 dispose de 32GB, c'est largement suffisant pour héberger tous les services.

2. Découverte du serveur VMS de démonstration

2.1.

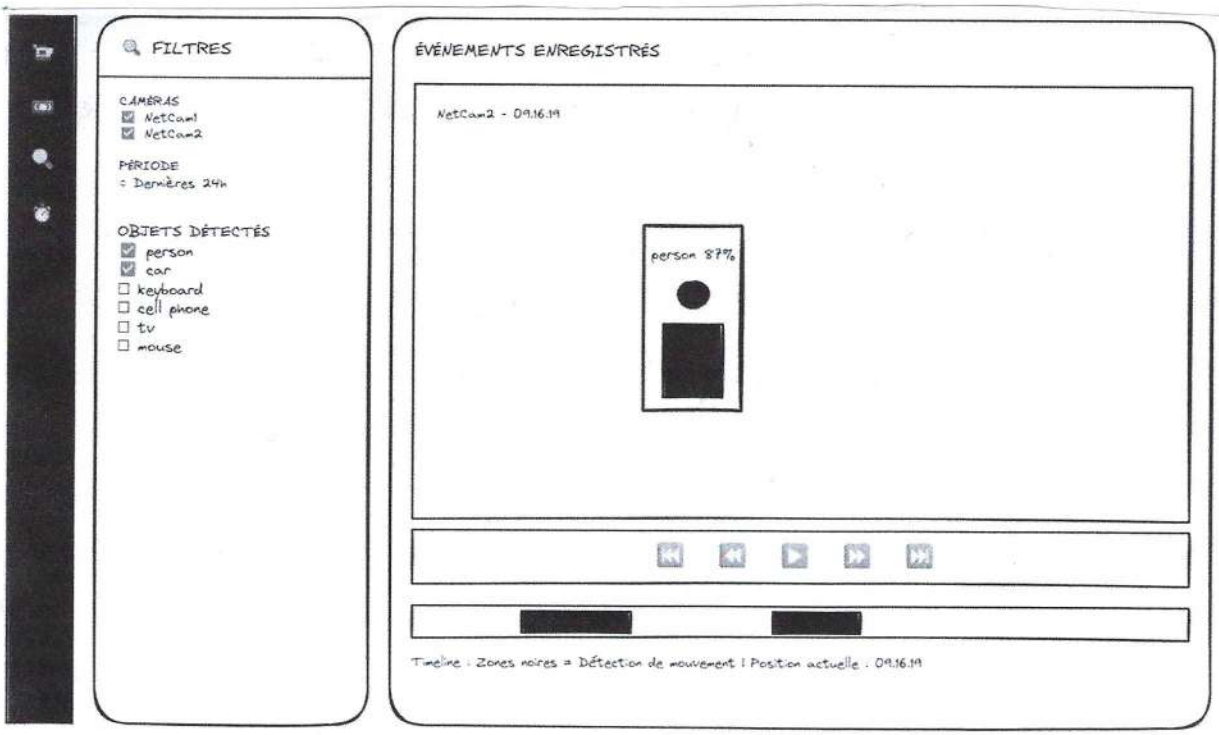
J'ai accédé au serveur de démonstration Frigate. L'URL https://demo.frigate.video m'a permis de découvrir l'interface. J'observe la vue principale affiche plusieurs éléments importants. A gauche, un menu de navigation donne accès aux fonctions. Le centre montre les flux vidéo en direct. Chaque caméra apparaît dans une grille. Les détections actives s'affichent avec des rectangles colorés.
Ça ressemble aussi à mon interface après installation :



2.2.

J'ai exploré la section "Review" de Frigate. J'observe une vue centralisée tous les évènements enregistrés. L'interface affiche une timeline horizontale. les évènements s'organisent par date et heure. Chaque évènement possède une miniature. Un clic sur la vignette ouvre le lecteur vidéo.

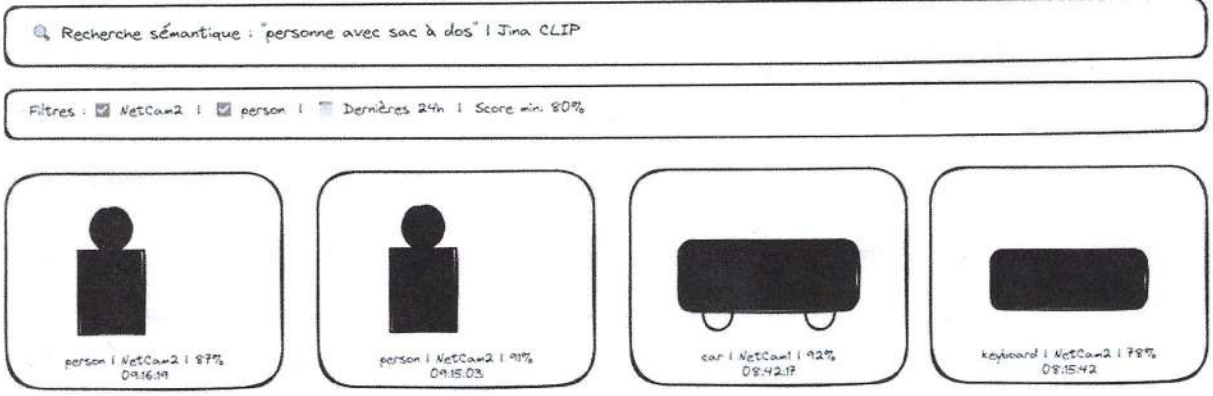
Le lecteur permet de naviguer dans l'enregistrement. les contrôles classiques sont présents. La timeline montre les zones de mouvement détecté. les filtres facilitent la recherche. Je peux filtrer par caméra, les types sont sélectionnables. Person, car, ---. La période temporelle se personnalise.



FONCTIONNALITÉS DE LA VUE REVIEW :
- Filtrage par caméra, période et type d'objet
- Lecteur vidéo avec contrôles classiques (play, pause, avance/recul rapide)
- Timeline interactive montrant les zones de détection de mouvement

2.3.

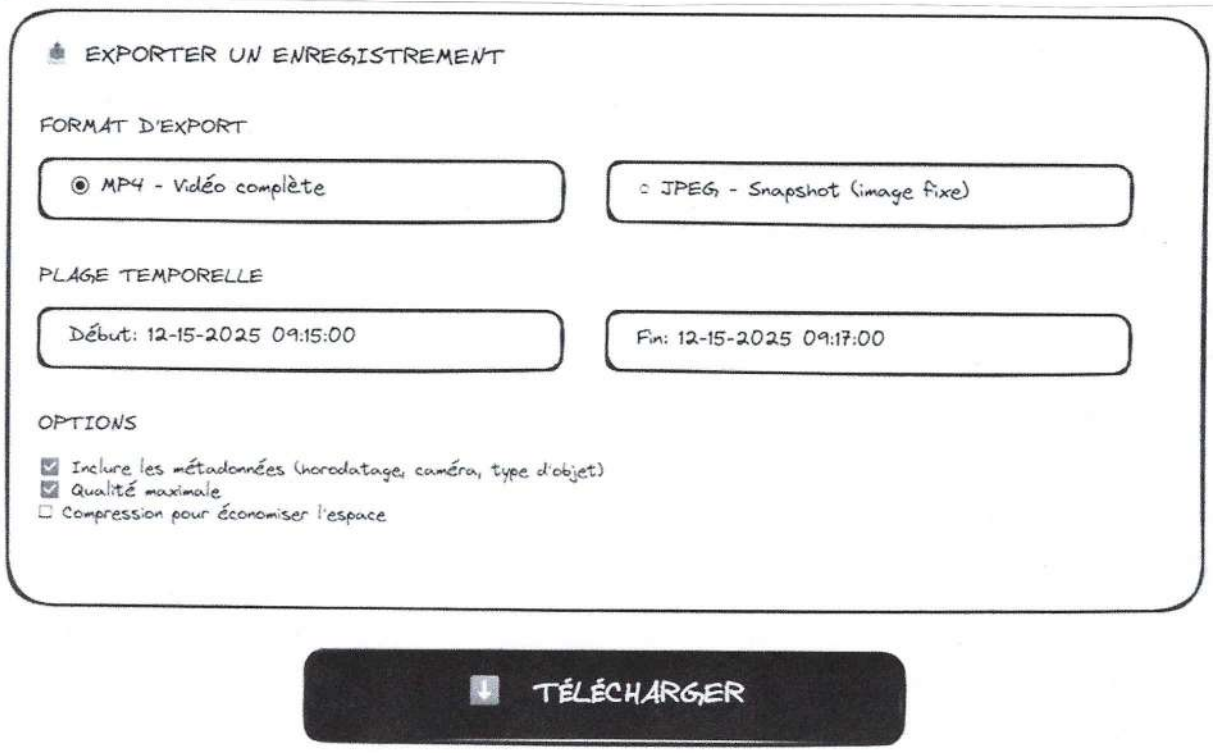
J'ai exploré la section "Explore" de Frigate. j'observe une vue affiche une grille de vignettes. Chaque vignette capture un objet détecté les infos. s'affichent clairement. Type d'objet, caméra source, horodatage et score de confiance. La recherche sémantique permet des requêtes en language naturel. "Personne avec sac" ou "voiture rouge" fonctionnent directement. le modèle CLIP comprend mes demandes. les filtres combinent caméra, période et type d'objet. Cette interface transforme la vidéosurveillance en outil analytique puissant.



EXPLORATION : Chaque vignette affiche un objet détecté avec son type, caméra source, score de confiance et horodatage.
Recherche sémantique via Jina CLIP : requêtes en langage naturel ("personne avec sac", "voiture rouge").

2.4.

J'ai testé la fonction "Export", j'observe que ça permet d'extraire des preuves vidéo. L'interface propose deux formats principaux, MP4 pour les vidéos complètes. JPEG pour les snapshots individuels.



EXPORT : Sélection du format (MP4/JPEG), plage temporelle, options métadonnées.
Téléchargement direct pour archivage ou partage.

3. Installation d'un serveur VMS local

Avant d'installer Frigate, j'ai rencontré un problème : Docker n'était pas disponible dans les dépôts standard de Debian 12.

→ nano /etc/apt/sources.list, j'ai remplacé "bookworm" par "Trixie" dans le fichier. Trixie est une version plus recente de Debian. Elle contient Docker dans ses dépôts officiels.

→ apt update, mise à jour de la liste des paquets
→ apt install docker.io
→ apt install docker-compose } installations de Docker et Docker Compose.

→ docker --version, ce que j'observe : Docker version 26.1.5+dfsg1, build a7...
→ docker compose version, ce que j'observe : Docker Compose version 2.26.1-4

Docker fonctionne. La version 26.1.5 est récente. Elle supporte toutes les fonctionnalités de Frigate. Docker compose est opérationnel.

- J'ai créé la structure des dossiers : Deux dossiers principaux : un pour la configuration, un pour les données.

→ mkdir -p /home/total/frigate, créé le dossier principal et tous les parents manquant
→ mkdir -p /home/total/frigate/configuration, dossier de configuration
→ mkdir -p /home/total/frigate/stockage, dossier de stockage.
→ cd /home/total/frigate.

- A cette étape, je crée la configuration de base :

→ nano /home/total/frigate/configuration/config.yml

```
mqtt:                    Désactivation de MQTT, protocole de messagerie non utilisé
  enabled: false         Je n'utilise pas de système domotique
cameras:                 Déclaration des caméras IP
  NetCam1:               Première caméra Hikvision
    enabled: true        Active cette caméra
    ffmpeg:
      inputs:
        Flux RTSP principal de la caméra
        - path: rtsp://admin:HikVision@10.129.8.44:554/Streaming/Channels/101
          admin:HikVision = identifiants de connexion
          10.129.8.44 = adresse IP de la caméra
          554 = port RTSP standard
          /Streaming/Channels/101 = flux principal haute qualité
          roles:
            - detect  # Cette caméra servira à la détection (plus tard)
  NetCam2: Deuxième caméra Hikvision
    enabled: true        Active cette caméra
    ffmpeg:
      inputs:
        Flux RTSP de la deuxième caméra
        - path: rtsp://admin:HikVision@10.129.8.48:554/Streaming/Channels/101
          roles:
            - detect     Cette caméra servira aussi à la détection
```

Ici, je teste d'abord la connexion aux caméras, et à l'interface web.

Le protocole RTSP fonctionne en client-serveur. les caméras sont les serveurs. Frigate est le client. L'URL contient tout : identifiants, adresse, port, chemin du flux.

- Je crée le fichier docker-compose.yml, Docker Compose orchestre le déploiement. Un fichier YAML définit tout : image, volumes, ports, réseau.

→ nano /home/total/frigate/docker-compose.yml • j'ai :

```
version: "3.9"                    Version 3.9 du format (compatible avec Docker Engine 19.03+)
services:                         Définition des services, un seul ici : Frigate
 frigate:
   container_name: frigate        Nom du conteneur, visible dans docker ps
     privileged: true             Mode privilégié : accès complet au matériel de l'hôte
   restart: unless-stopped        Le conteneur redémarre toujours sauf arrêt manuel
   image: ghcr.io/blakeblackshear/frigate:stable   Image Docker officielle de Frigate, ghcr.io =
GitHub Container Registry, blakeblackshear = créateur de Frigate, stable = version stable (pas de
développement)
   shm_size: "256mb"              Mémoire partagée allouée au conteneur, 256 Mo suffisent
pour 2 caméras en 720p, Frigate utilise cette mémoire pour traiter les images
   volumes:   Montage des volumes (liaison dossiers hôte ↔ conteneur)
     - ./configuration:/config    Configuration : ./configuration (hôte) → /config (conteneur),
Frigate lit  son fichier config.yml depuis /config
     - ./storage:/media/frigate   Stockage : ./storage (hôte) → /media/frigate (conteneur),
Frigate écrit ses enregistrements dans /media/frigate
     - /etc/localtime:/etc/localtime:ro   Synchronisation de l'heure système, :ro = lecture seule
(read-only)
   ports:                         Exposition des ports (hôte:conteneur)
     - "5000:5000"                Port 5000 : interface web Frigate, Accessible via
http://10.129.8.15:5000
     - "8554:8554"                Port 8554 : serveur RTSP interne de Frigate, Permet de re-streamer
les flux caméras
     - "8555:8555/tcp"            Port 8555 : WebRTC pour streaming temps réel
     - "8555:8555/udp"            TCP et UDP nécessaires pour la compatibilité
   environment:                   Variables d'environnement passées au conteneur
     FRIGATE_RTSP_PASSWORD: "HikVision"   Mot de passe pour accéder aux flux RTSP de Frigate
   networks:                      Réseau Docker utilisé
     - default
networks:                         Définition du réseau Docker
 default:
   name: frigate_default          Nom explicite du réseau créé, Facilite l'identification et le débogage
```

Ce fichier a tout le déploiement. Docker Compose lit ce fichier. Il crée automatiquement le conteneur. Les volumes lient les dossiers. Les ports sont exposés. Le réseau permet la communication.

le mode privilégié est nécessaire pour que Frigate accède au processeur pour la détection, sans ce mode, certaines fonctionnalités échoueraient. La mémoire (shm) sert au traitement d'images.

- le déploiement se fait en une commande. Docker compose gère tout

: docker compose up -d
↳ utilise docker compose
↳ mode détaché, en arrière-plan
↳ démarre les services définis dans docker-compose.yml

J'observe :

```
[+] Running 1/1
 ✓ Container Frigate Started.
```

Après le lancement, je vérifie que tout fonctionne correctement.

→ docker compose ps

Ce que j'observe: NAME IMAGE COMMAND ...

frigate ghcr.io/blakeblackshear/frigate:stable "/init" ...

→ docker inspect frigate | grep IPAddress

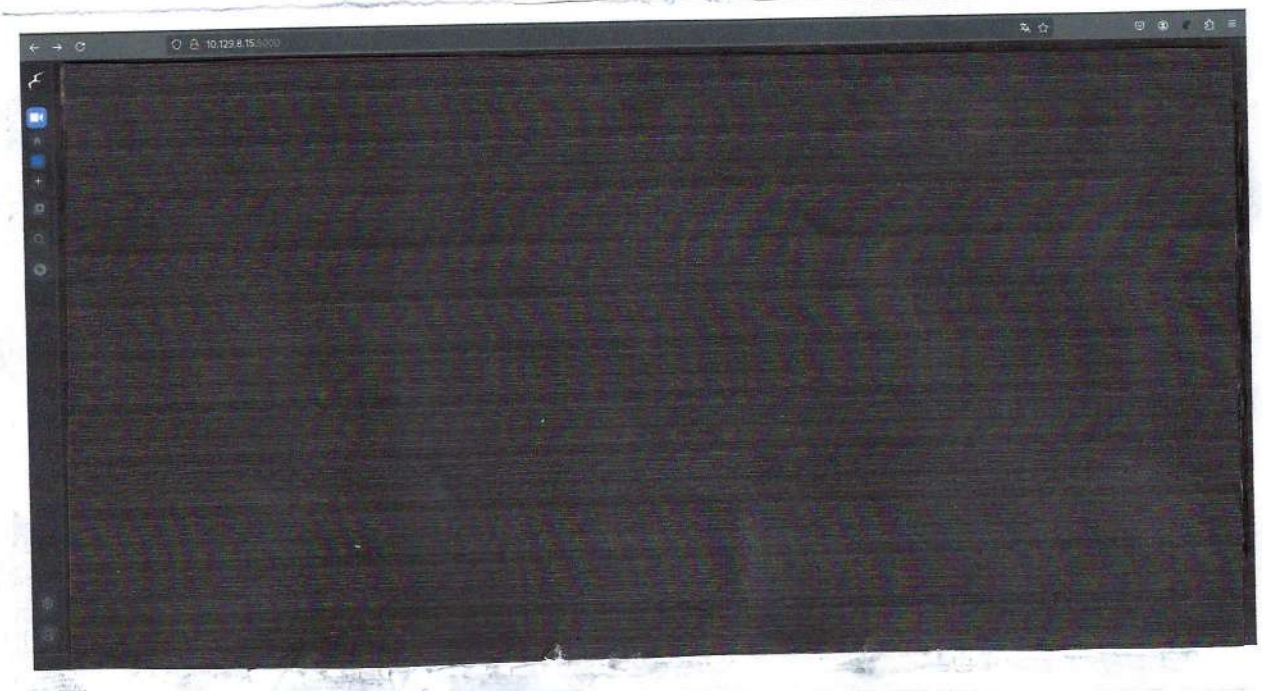
Ce que j'observe: ...

```
"SecondaryIPAddresses": null,
"IPAddress": "",
"IPAddress": "172.18.0.2"
```

Le conteneur a l'IP 172.18.0.2 sur le réseau Docker frigate_default. Cette IP permet la communication inter-conteneurs. Mon serveur hôte a IP 10.129.8.15 sur le réseau local. C'est cette IP que j'utiliserai pour accéder à l'interface web.

- Maintenant que Frigate tourne, je peux accéder à son interface. J'ouvre Firefox et je navigue vers http://10.129.8.15:5000

Ce que j'observe: Frigate fonctionne !



L'interface Frigate se charge. Je vois la page d'accueil. L'onglet "Live" est sélectionné par défaut. Aucune caméra configurée, donc, aucun flux, rien n'est affiché. Mais Frigate fonctionne !

4. Exploitation basique du serveur VMS local

4.1. Visualisation de caméra

1. Pour afficher le flux vidéo de ma première caméra IP, j'ai configuré une section dans le fichier config.yml de Frigate, la configuration de la caméra NetCam 1, j'indique le chemin du flux, la résolution et le nombre d'images par seconde :

```
cameras:
  NetCam1:
    enabled: true
    ffmpeg:
      inputs:
        - path: rtsp://admin:HikVision@10.129.8.44:554/Streaming/Channels/101
          roles:
            - detect
            - record
    detect:
      enabled: true
      width: 1280
      height: 720
      fps: 5
```

j'ai activé la caméra enabled : true

L'URL RTSP contient les identifiants de connexion ; admin → utilisateur ; HikVision → mot de passe

/101 → Correspond au flux principal de caméra.

detect → signifie que le flux sert à la détection d'objets.

record → record pour pouvoir enregistrer cette caméra.

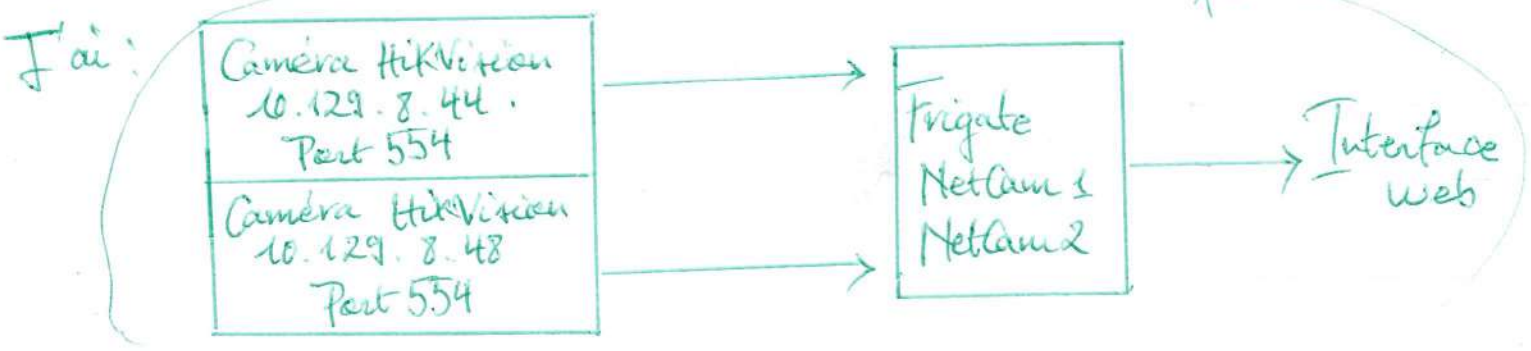
La résolution est de 1280X720 pixels à 5 images par seconde.

2. Pour ma deuxième caméra, son adresse IP est 10.129.8.48 ;

```
cameras:
  NetCam2:
    enabled: true
    ffmpeg:
      inputs:
        - path: rtsp://admin:HikVision@10.129.8.48:554/Streaming/Channels/101
          roles:
            - detect
            - record
    detect:
      enabled: true
      width: 1280
      height: 720
      fps: 5
```

:554 → le port standard RTSP.

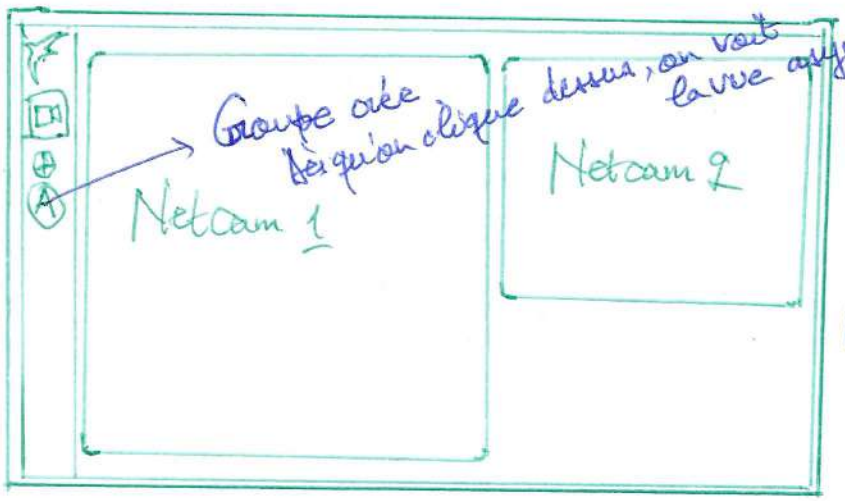
J'ai :



J'observe la caméra Netcam 2 apparaît à côté de Netcam 1 dans l'interface web. Elles montrent le flux en direct. Je vois la table en temps réel. Le compteur FPS indique 5 images par seconde.

3. Pour créer une vue avec la première caméra en grand à gauche et la seconde en petit à droite. Dans l'interface, à gauche, dans la "vue direct", j'ai crée un groupe de caméras "+". J'ai entrée le nom et les caméras voulues, j'ai mis une icône et j'ai enregistré. Enfin, j'ai glissé les côtés d'une caméras pour l'agrandir et avoir une vue asymétrique.

J'observe à l'interface :

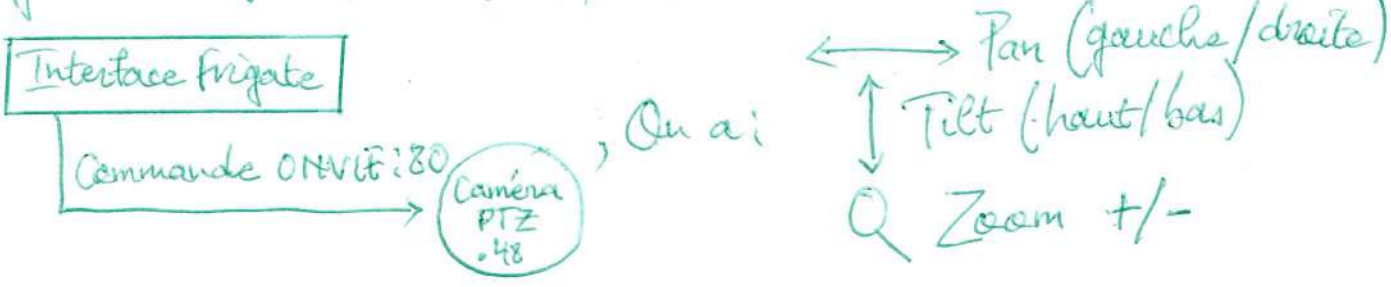


Et donc, dans le menu de gauche, je vois le groupe "Vue_asymetrique". Quand je clique dessus, les deux caméras s'affichent côte à côte. Netcam 1 prend plus de place que Netcam 2.

4. Pour contrôler les mouvements de la caméra (Pan, Tilt, Zoom), j'ai configuré le protocole ONVIF. Ce protocole permet de communiquer avec les caméras IP compatibles : → nano /home/toto/frigate/configuration/config.yml, j'ajoute dans la cam 2 :

```
Cameras :
  Netcam 2
  ---
    onvif :
      host : 10.129.8.48 , IP de la caméra.
      Port : 80 , le port HTTP standard pour ONVIF
      User : admin
      password : HikVision } paramètres de connexion. les identifiants sont les mêmes que pour le flux RTSP.
```

J'observe dans l'interface, dans la caméra :



4.2. Paramétrage des enregistrements.

1. Par défaut, Frigate peut enregistrer en continu toutes les caméras, j'ai d'abord configuré la section globale "record" pour définir les paramètres d'enregistrement : J'ajoute en bas dans le code de config, config.yml :

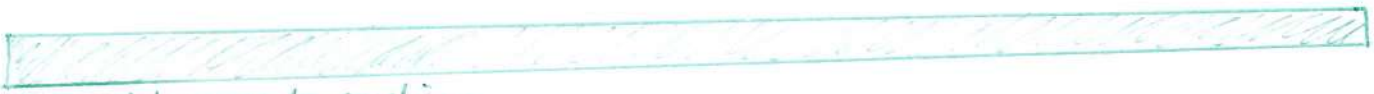
```
record:
  enabled: true , ce paramètre active l'enregistrement
  retain:
    days: 7 , le paramètre "retain.days:7" garde les vidéos pendant 7 jours.
    mode: all , le mode all signifie que Frigate enregistre tout le temps,
                même sans détection. C'est l'enregistrement permanent.
```

Pour que NetCam 2 enregistre, j'ai ajouté le rôle record dans ses entrées :

```
cameras:
  NetCam2:
    ffmpeg:
      inputs:
        - path: rtsp://admin:HikVision@10.129.8.48:554/Streaming/Channels/101
          roles:
            - detect
            - record → le rôle 'record' indique ce flux sert aussi à l'enregistrement.
```

Enregistrement permanent (mode all) :

Temps →



Enregistrement continu

Stockage : Beaucoup d'espace disque utilisée.
Retention : 7 jours puis suppression automatique.
J'observe dans l'interface Frigate, dans "la revue d'evenements" dans le menu de gauche, on choisit un évenement, je vois une timeline continue pour NetCam2. Les segments vidéo sont affichés en gris. Je peux cliquer sur n'importe quel moment pour relire la vidéo. L'espace disque se remplit rapidement avec ce mode.

2. Pour économiser l'espace disque, j'ai modifié la configuration pour enregistrer seulement quand il y a une détection d'objet. J'ai changé le mode et mis la rétention continue à 0 :

```
record:
  enabled: true
  retain:
    days: 0
  mode: active_objects
alerts:
  retain:
    days: 14
detections:
  retain:
    days: 14
```

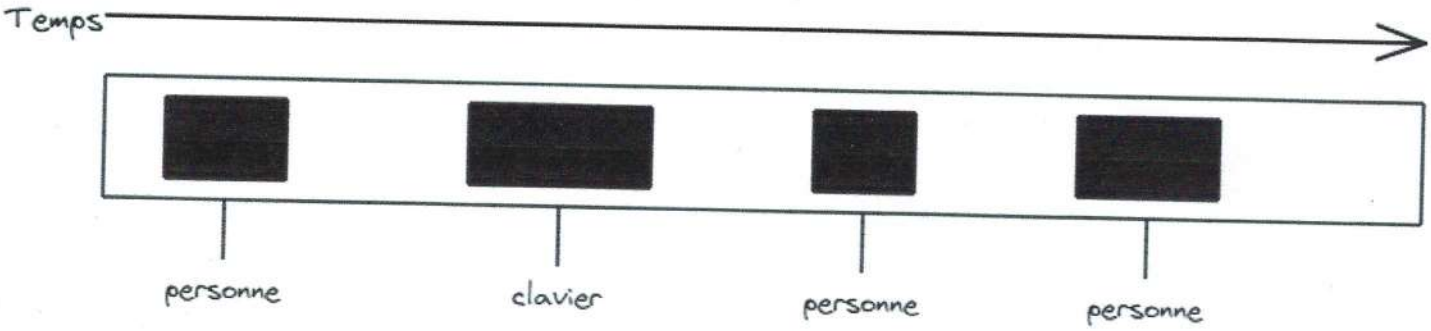
désactive l'enregistrement continu.

le mode "active_objects" enregistre uniquement quand un objet est détecté.

garde les alertes importantes pendant 14 jours.

garde les clips de détection pendant 14 jours aussi.

ENREGISTREMENT SUR DETECTION (mode: active_objects)



Legende :
- ■ = Enregistrement (detection)
- □ = Pas d'enregistrement

Stockage : Reduit
Retention : 14 jours (detections)
Mode : active_objects

COMPARATIF DES MODES D'ENREGISTREMENT

MODE : all

(Enregistrement permanent)

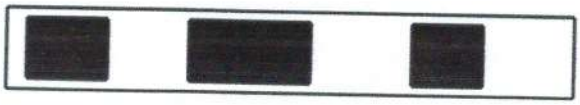


Enregistre : TOUT le temps
Espace disque : ELEVE
retain.days : 7

VS

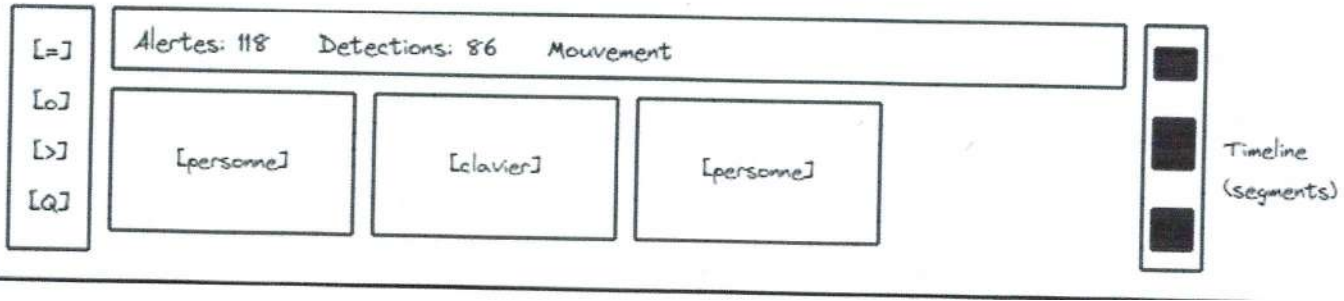
MODE : active_objects

(Enregistrement sur detection)



Enregistre : Si DETECTION
Espace disque : REDUIT
retain.days : 0 + detections: 14

VUE INTERFACE FRIGATE



J'observe alors dans la timeline, je vois des segments colorés aux moments où Frigate a détecté quelque chose. Les zones sans détection sont vides. Quand je passe devant la caméra, un nouveau segment apparaît. L'espace disque utilisé est beaucoup plus faible qu'avec le mode permanent.

5. Exploitation avancée du serveur VMS : Analyses vidéo

5.1. Détection d'objets

1. Frigate utilise des détecteurs pour analyser les images et trouver des objets. J'ai configuré deux détecteurs CPU pour répartir la charge de travail :

```
detectors:
  cpu1:
    type: cpu
    num_threads: 3
  cpu2:
    type: cpu
    num_threads: 3
```

J'ai orée deux détecteurs comme Cpu et 2. Le paramètre type: cpu indique que j'utilise le processeur pour l'inférence.

Le paramètre "num_threads : 3" alloue 3 fils d'exécution à chaque détecteur. Avec 2 détecteurs, j'ai donc 6 threads total pour l'analyse.

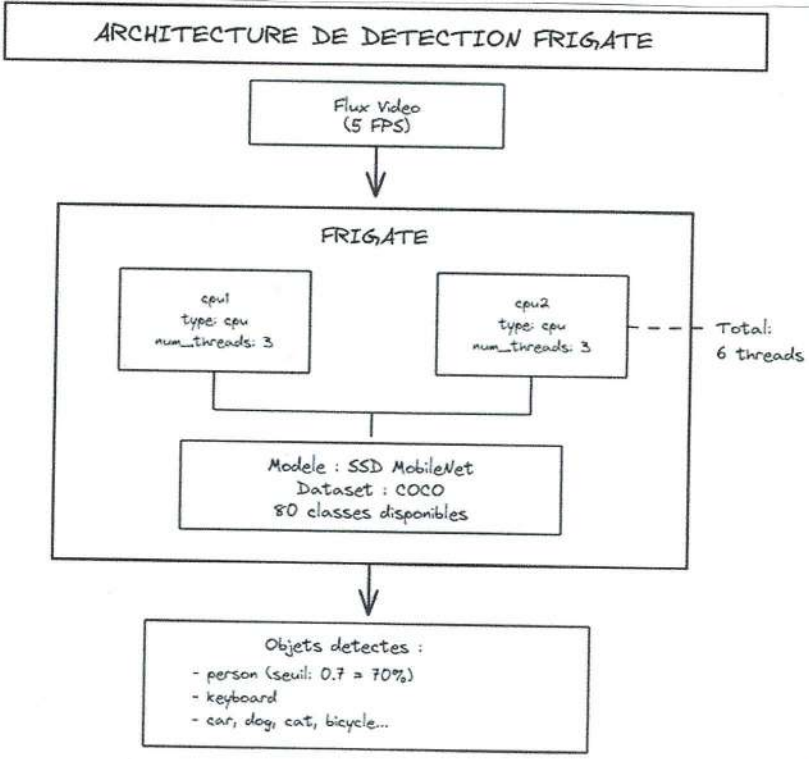
Par défaut, Frigate utilise le modèle SSD MobileNet entraîné sur le dataset COCO. Ce modèle peut détecter 80 classes d'objets différentes comme : person, car, dog, cat, bicycle, etc.

Ensuite, j'ai défini les objets à détecter dans le fichier de config :

```
objects:
  track:
    - person
    - keyboard
  filters:
    car:
      min_area: 500
      threshold: 0.6
    person:
      min_area: 1000
      threshold: 0.7
...
```

La liste "track" contient les objets que Frigate doit suivre.
Pour détecter les personnes
" —— " — les claviers.
Cette section permet d'affiner la détection
Ce paramètre définit la taille minimale en pixels.
celui-ci le seuil de confiance entre 0 et 1.
Une personne doit avoir un score d'au moins 70% pour être détectée.

J'observe dans l'interface Frigate, je vois en bas à gauche "CPU 15%." qui indique l'utilisation des détecteurs.



Quand une personne passe devant la caméra NetCam2, un rectangle vert apparaît autour d'elle avec le label "person" et le pourcentage de confiance.

2. Les bounding boxes sont les rectangles qui entourent les objets détectés. J'ai activé leur affichage dans les snapshots :

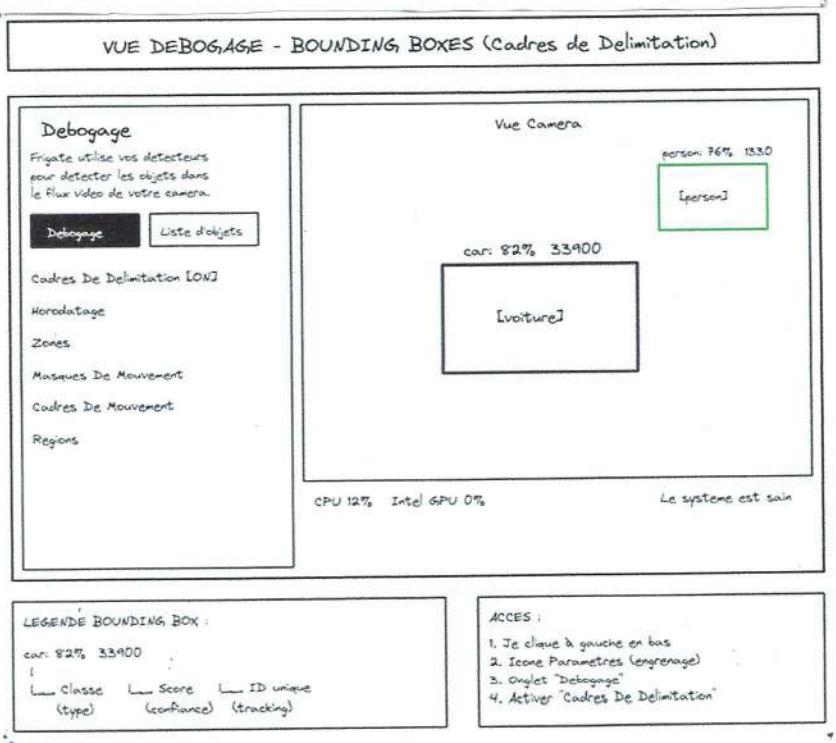
```
snapshots:
  enabled: true
  retain:
    default: 14
  bounding_box: true
```

→ garde les snapshots pendant 14 jours.

le paramètre bounding_box: true active l'affichage des rectangles de détection sur les captures d'écran.

```
cameras:
  NetCam1:
    snapshots:
      enabled: true
  NetCam2:
    # snapshots hérite de la config globale
```

à l'interface : Paramètres → Débogage :



Dans l'onglet "Débogage", je vois l'image capturée avec le rectangle autour de l'objet. Le rectangle est vert pour les personnes. Le label affiche le type d'objet et le score de confiance.

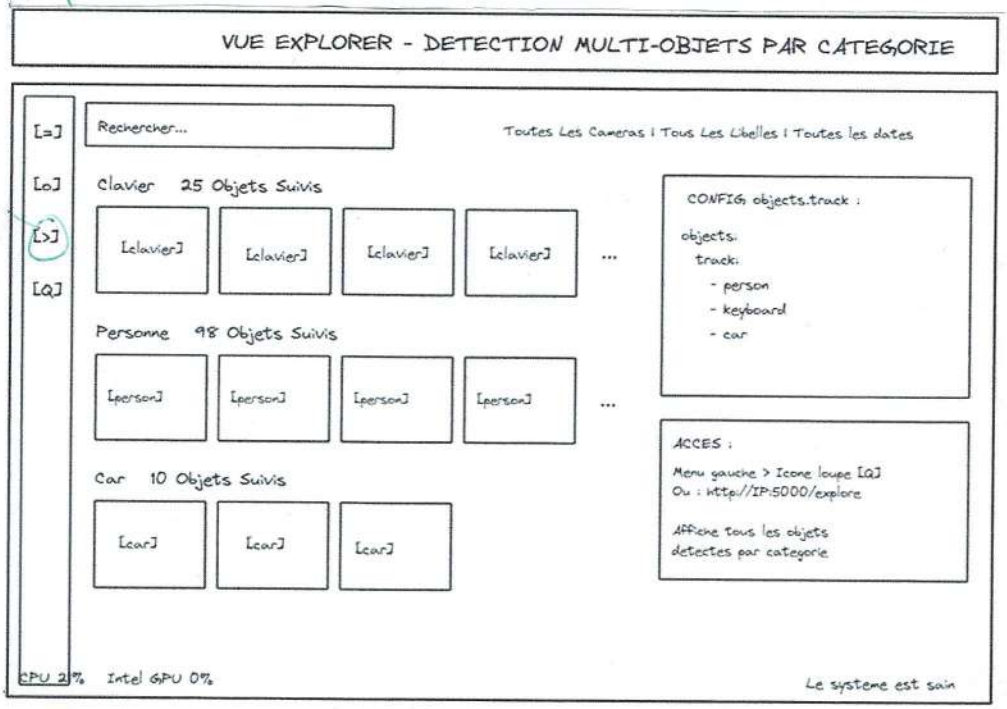
Par exemple : "Person 85%" signifie que Frigate est sûr à 85% que c'est une personne.

3. Le modèle CoCo de Frigate peut détecter 80 objets différents. Pour détecter un clavier, j'ai simplement ajouté "Keyboard" dans la liste des objets à suivre. J'ai configuré cela uniquement sur NetCam2 :

```
cameras:
 NetCam2:
  objects:
   track:
    - person
    - keyboard
```

→ la section "objects.track" au niveau de la caméra définit les objets spécifiques à cette caméra. NetCam2 détecte les personnes et les claviers.

Ce que j'observe dans le menu à gauche sur l'icône loupe [Q] :



les objets détectés.

Et dans l'onglet debogage du paramètre, on peut voir Frigate dessinant un rectangle autour du clavier avec le label "keyboard".

5.2. Lecture de plaques minéralogiques.

1. Pour mettre en oeuvre la lecture de plaques minéralogeiques, j'ai d'abord utilisé une vraie caméra IP : 10.129.8.44. Je l'ai positionné en direction de la plaque de test RT-1994-LP fournie.

J'ai utilisé les contrôles PTZ via le protocole ONVIF pour orienter la camera.

```
cameras:
 NetCam1:
  onvif:
   host: 10.129.8.44
   port: 80
   user: admin
   password: HikVision
```

→ La section onvif permet de communiquer avec la cam pour la contrôler.

} Authentification.

Dans l'interface Frigate, j'ai cliqué sur la caméra NetCam1 pour l'afficher en plein écran. En bas de l'écran, les boutons PTZ sont apparus : ← → ↑ ↓ pour le mouvement, et les boutons + et - pour le zoom. J'ai utilisé ces contrôles pour orienter la caméra vers la plaque RT-1994-LP jusqu'à ce qu'elle soit bien visible au centre de l'image.

2. Pour activer la reconnaissance de plaques, j'ai configuré la section LPR (License Plate Recognition) globalement et sur la NetCam1 :

```
lpr:
  enabled: true
  device: CPU
  detection_threshold: 0.7
  recognition_threshold: 0.8
  min_area: 500
```

- `enabled: true` → active la fonctionnalité LPR.
- `device: CPU` → utilise le processeur pour l'analyse.
- `detection_threshold: 0.7` → le seuil signifie que Frigate doit être sûr à 70 % qu'il y a une plaque.
- `recognition_threshold: 0.8` → ce seuil demande 80 % de confiance pour lire les caractères.
- `min_area: 500` → définit la taille minimale de la plaque en pixels.

Ensuite, j'ai configuré NetCam1 comme caméra dédiée LPR :

```
cameras:
  NetCam1:
    type: lpr
    enabled: true
    lpr:
      enabled: true
      enhancement: 3
    objects:
      track: []
...
```

- `type: lpr` → indique que cette caméra est spécialisée pour la lecture de plaques.
- `enabled: true` → active la reconnaissance sur cette caméra.
- `enhancement: 3` → ce paramètre 'enhancement: 3' améliore la netteté de l'image pour mieux lire les caractères
- `track: []` → la liste est vide car je ne veux pas détecter d'autres objets sur cette caméra.

Après avoir redémarré Frigate, la caméra 1 affiche la plaque de test, j'observe

